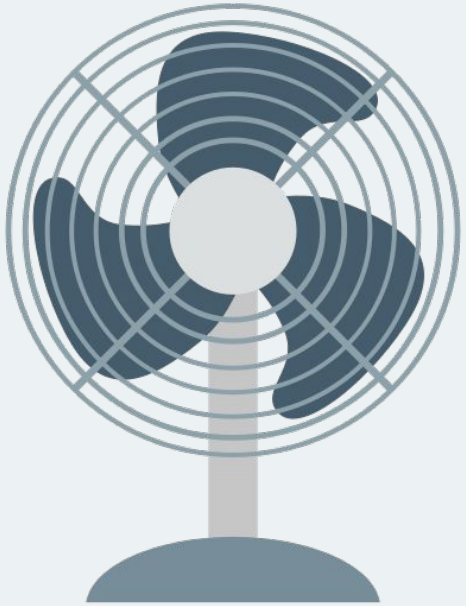
AI시스템반도체 SW개발자 2기

Linux 커널 드라이버 기반 스마트 선풍기

GPIO · PWM · I²C 주변장치 통합



김형진, 현수근, 신유지,
도재우

개발 목표

커널 드라이버 6개 (입력 - 버튼·엔코더, 모터, 서보 모터, LED Bar, 온도 센서, LCD) + 유저 공간 제어 앱

커널 드라이버

- 버튼 · 엔코더
- 모터
- 서보 모터
- **LED Bar**
- **BMP180**
- **LCD**

어플리케이션

- **전원**
  전원 ON/OFF 시 풍량, 회전 상태 초기화
- **회전 기능**
  회전 버튼으로 좌우 회전 ON/OFF
- **수동 모드 / 자동 모드**
  수동(MAN)은 엔코더, 자동(AUTO)은 온도로 풍량 결정
- **타이머**
  엔코더로 시간 설정(최대 12시간), 카운트다운 후 종료

FSM

Power FSM 전원 버튼



OFF: 풍량 0, 회전 OFF, MAN 복귀, 타이머 초기화
ON: 풍량 1단계로 시작

Mode FSM 풍량 엔코더 누름



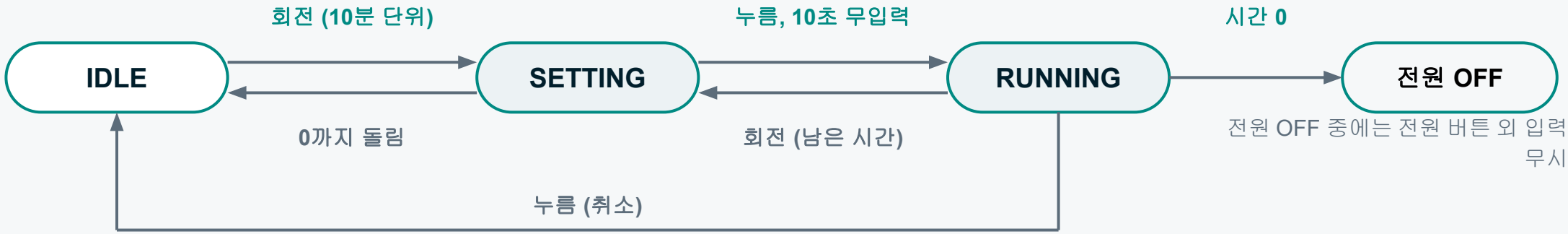
MAN: 엔코더 회전으로 1~4단계
AUTO: 온도로 0~4단계, 엔코더 회전 무시

Swing FSM 회전 버튼



ON: 25ms마다 1°, 30~150° 왕복
OFF: 현재 각도에서 정지

Timer FSM 타이머 엔코더

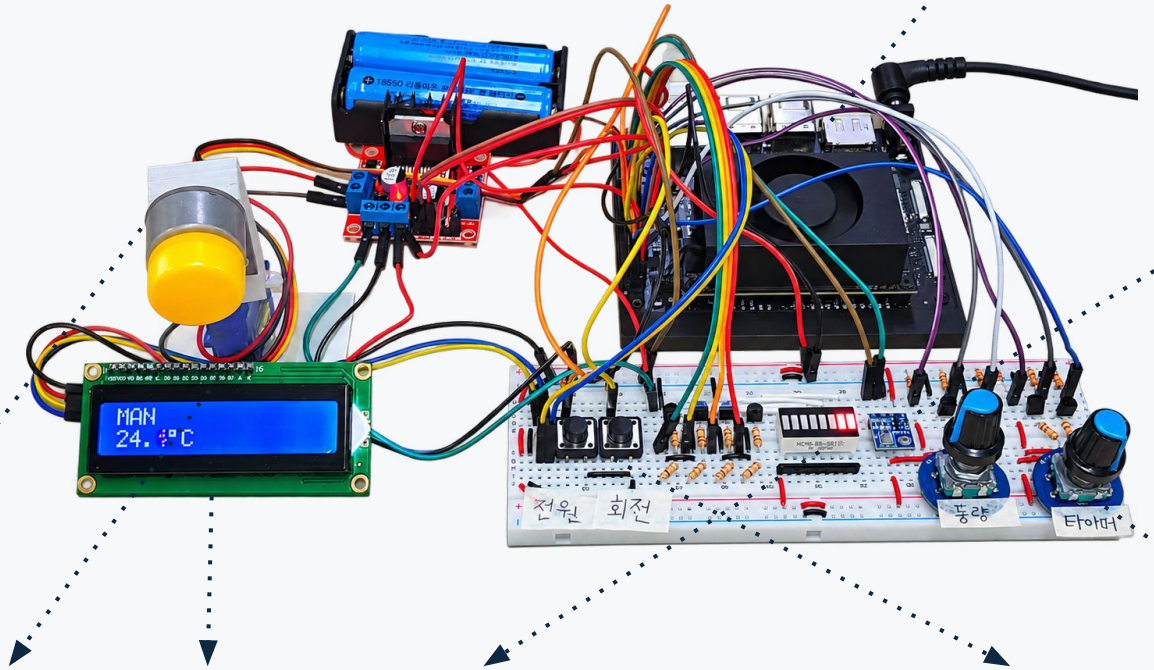


전원 OFF 중에는 전원 버튼 외 입력 무시

시스템 구성과 제어 흐름

입력 → 이벤트 → 상태 결정 → 주변장치 출력

처리: 입력 / 타이머 / 센서 스레드와 공유 상태



측정: BMP180 온도
→ 자동 풍량 정책

출력: DC 모터 · 서보 · LCD1602 · LED 바 4단계

입력: 전원·회전 버튼 + 풍량·타이머 엔코더

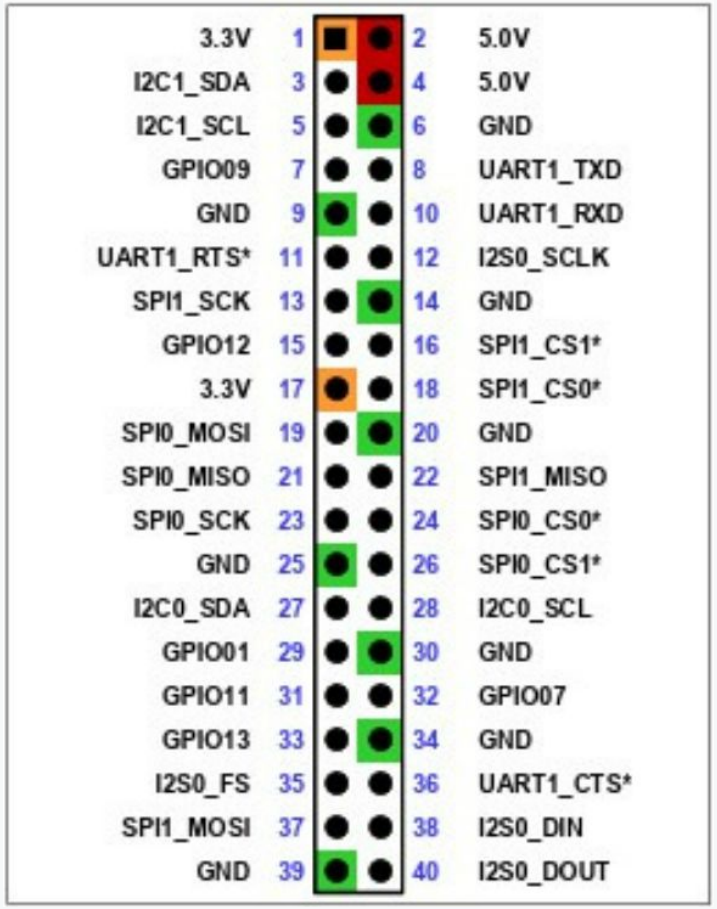
파트 리스트

부품	수량	용도
Jetson Orin Nano 개발자 키트	1	메인 보드, 드라이버와 앱 실행
DC 모터 (R300)	1	송풍
L298N 모터 드라이버	1	모터 구동, 서보용 5V 공급
서보 모터 (SG90)	1	좌우 회전
로터리 엔코더	2	풍량과 모드, 타이머
택트 스위치	2	전원, 회전
8bit LED 바 + 저항 배열	1	풍량 단계 표시
NPN 트랜지스터 MPS2222A	4	LED 구동
BMP180 모듈	1	온도 측정 (I²C)
LCD1602 + PCF8574T	1	상태 표시 (I²C)
저항 330 Ω	-	버튼, 엔코더 풀업
배터리 (≒ 8V)	1	모터, 서보 전원



핀 배정

기능	헤더 핀
모터 **PWM** / 서보 **PWM**	32 / 33
모터 방향 / **LED** 바 **4**단계	37, 40 / 13, 16, 18, 22
전원 버튼 / 회전 버튼	29 / 31
BMP180 / LCD	3, 5 / 27, 28
풍량 엔코더 **KEY / S2 / S1**	7 / 11 / 36
타이머 엔코더 **KEY / S2 / S1**	38 / 12 / 35



Jetson Orin Nano 핀 맵

주변장치와 장치 파일

통신 방식에 따른 사용자 인터페이스

주변장치	커널 인터페이스	사용자 공간
엔코더 · 버튼	**GPIO IRQ + FIFO**	/dev/fan_input · read/poll
DC 모터 / 서보 모터	**PWM + GPIO / PWM**	/dev/fan_motor · /dev/fan_servo · ioctl
LED 바 (4단계)	**GPIO descriptor 배열**	/dev/fan_led · ioctl
BMP180	**I²C (0x77)**	/dev/bmp180 · read/lseek
LCD1602 + PCF8574T	**I²C (0x27)**	/dev/lcd1602 · pwrite/ioctl

include/fan_ioctl.h · driver/fan_input/fan_event_queue.c

Device Tree와 드라이버 연결

compatible 매칭 후 probe에서 하드웨어 자원 획득

```
fan_motor {                                     /* C */
    compatible = "kcci,fan-motor";
    pwms = <&pwm7 0 50000>;                     /* 20 kHz */
    dir-gpios = <&gpio TEGRA234_MAIN_GPIO(Y, 2) GPIO_ACTIVE_HIGH>,    /* IN1 */
                <&gpio TEGRA234_MAIN_GPIO(I, 0) GPIO_ACTIVE_HIGH>;    /* IN2 */
    pinctrl-names = "default";
    pinctrl-0 = <&fan_motor_pins>;
```

pwms: 컨트롤러·채널·주기 [ns]

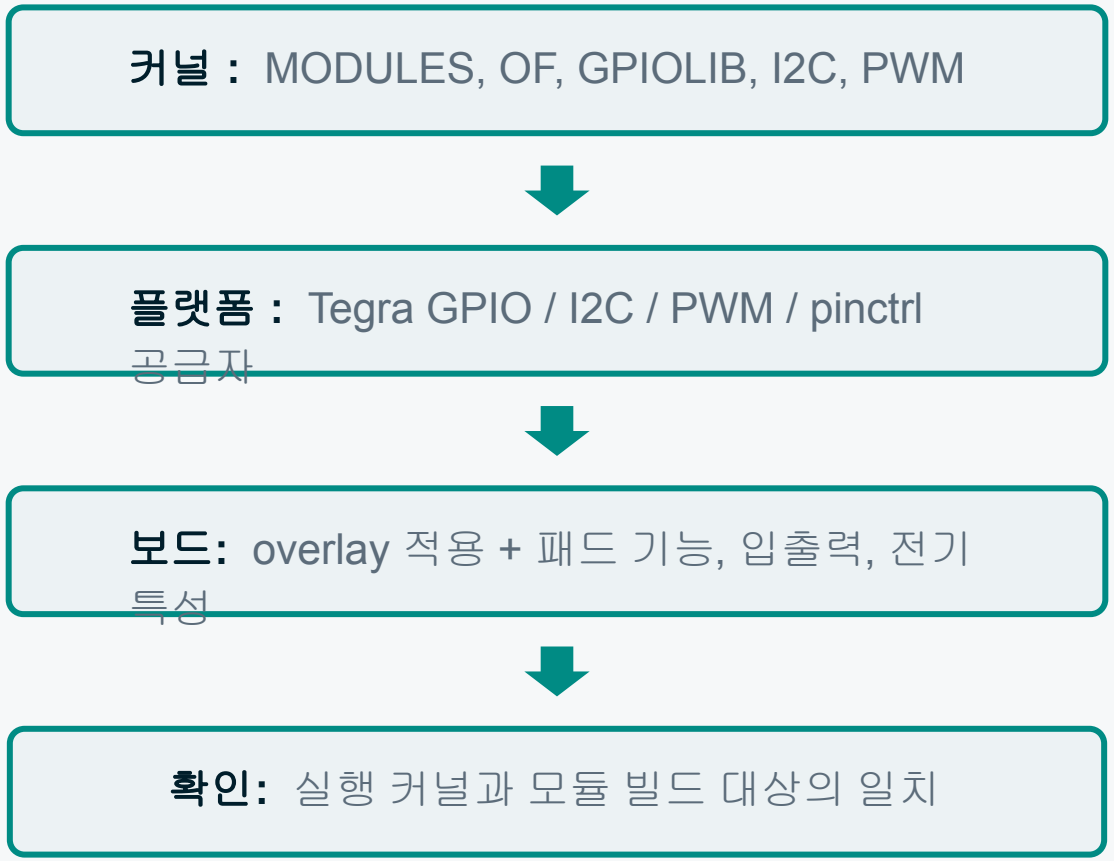
dir-gpios: 모터 방향 GPIO 두 개 (IN1, IN2)

probe: PWM → GPIO → 초기 출력 → misc 등록

dts/fan-overlay.dts:37–43

커널 설정과 보드 설정

설정 확인: 빌드 → DT → pinmux → probe 순서



dts/fan-overlay.dts · driver/Makefile

로터리 엔코더: 전이표로 회전 방향 판정

양쪽 에지를 수집하고 detent에서 ±1 이벤트 생성

```
    phase &= 3U;
    if (phase == decoder->previous)
        return 0;
    delta = transitions[(decoder->previous << 2) | phase];
    decoder->previous = phase;
    if (!delta) {
        decoder->accumulator = 0;
        decoder->synchronized = fan_encoder_at_detent(decoder, phase);
        return 0;
    }
```

```
/*
 * 이전 상태 → 현재 상태의 변화가 어느 방향인지 찾는 표.
 *
 *                    현재 A/B 상태
 *                    00   01   10   11
 * 이전 상태 00        0   +1   -1    0
 * 이전 상태 01       -1    0    0   +1
 * 이전 상태 10       +1    0    0   -1
 * 이전 상태 11        0   -1   +1    0
 *
 * +1: 정방향으로 한 단계 변화
 * -1: 역방향으로 한 단계 변화
 *  0: 같은 상태이거나 정상적인 한 단계 변화가 아님
 *
 * 여기의 +/-1은 아직 '딸깍 한 번'이 아니라
 * A/B 상태가 한 단계 바뀌었다는 의미.
 */
```

S1·S2를 2비트 상태로 구성

정상 전이 누적 → 2회 또는 4회에 한 클릭

인접 바운스는 상쇄, 두 비트 점프는 재동기화

driver/fan_input/include/fan_input_logic.h:64–73

버튼 IRQ와 지연 디바운스

IRQ에서 예약, workqueue에서 상태 확인

```
static irqreturn_t button_irq(int irq, void *data)
{
    struct button *b = data;

    mod_delayed_work(system_wq, &b→work, msecs_to_jiffies(debounce_ms));
    return IRQ_HANDLED;
}
```

양쪽 에지마다 20 ms 뒤로 **작업 재예약**
workqueue에서 sleep 가능한 GPIO 읽기
active-low 버튼도 **논리값 1이 눌림 상태**

driver/fan_input/button_input.c:39–45

이벤트 큐와 read, poll

커널에서 만든 이벤트를 앱이 읽어 가는 방법

- 512개 × 8바이트 이벤트 · spinlock으로 큐 보호
- poll(이벤트 및 상태 감지): 데이터 / overflow / 종료 상태 전달
- read: peek → copy_to_user → 성공한 이벤트 제거
- EOVERFLOW는 이벤트 손실, EAGAIN은 재시도

DC 모터: PWM 소비자 API

주기 50,000 ns = 20 kHz · 레벨별 듀티 적용

```c
static int motor_apply_duty(unsigned int percent)
{
    struct pwm_state st;

    pwm_init_state(motor_pwm, &st);
    pwm_set_relative_duty_cycle(&st, percent, 100);
    st.enabled = true;
    return pwm_apply_state(motor_pwm, &st);
}
```

```
fan_motor {
    compatible = "kcci,fan-motor";
    pwms = <&pwm7 0 50000>;
    dir-gpios = <&gpio ((18 * 8) + 2) 0>,
                <&gpio ((8 * 8) + 0) 0>;
    pinctrl-names = "default";
    pinctrl-0 = <&fan_motor_pins>;
};
```

레벨 0 / 1 / 2 / 3 / 4

듀티 0 / 45 / 60 / 80 / 100%

L298N 방향 GPIO와 PWM을 함께 제어

driver/motor/fan_motor_main.c:30–38

DC 모터: 모터 기동 보조와 비동기 복귀

정지 → 구동 전환 시 100% 듀티를 200 ms 요청

```c
static void motor_kick_end(struct work_struct *work)
{
    mutex_lock(&motor_lock);
    if (kicking) { //0 으로 꺼졌으면 할 일 없음
        kicking = false;
        motor_apply_duty(level_duty[cur_level]);
    }
    mutex_unlock(&motor_lock);
}
```

기동 토크 확보를 위한 **kickstart**

지연 작업이 최신 cur_level의 듀티로 복귀

mutex로 ioctl과 지연 작업의 상태 공유 보호

driver/motor/fan_motor_main.c:41–49

서보 모터: 서보 각도와 펄스 폭

각도를 펄스 폭으로 변경 → 500–2,500 µs 범위에 각도 0–180°를 대응

```c
static int servo_apply_pos(int deg)
{
    struct pwm_state st;
    u64 span_ns = (u64)(max_pulse_us - min_pulse_us) * 1000;

    pwm_init_state(servo_pwm, &st);      //DT의 PWM 주기로 채움
    st.duty_cycle = (u64)min_pulse_us * 1000 + div_u64(span_ns * deg, 180);
    st.enabled = true;
    return pwm_apply_state(servo_pwm, &st);
}
```

```
fan_servo {
    compatible = "kcci,fan-servo";
    pwms = <&pwm5 0 5000000>;
    min-pulse-us = <500>;
    max-pulse-us = <2500>;
    swing-min-deg = <30>;
    swing-max-deg = <150>;
};
```

90° → 1,500 µs · 회전 범위 30–150°

25ms마다 1°씩 이동, 끝에서 방향 반전

현재 DT 주기 5 ms = 200 Hz

driver/servo/fan_servo_main.c:35–44

LED: GPIO 배열과 논리 출력

풍량 단계에 따른 LED 점등

```c
static int led_show_level(int level){
    unsigned int i;

    if (level < 0 || level > FAN_LEVEL_MAX)
        return -EINVAL;
    mutex_lock(&led_lock);
    for (i = 0; i < leds->ndescs; i++)
        gpiod_set_value_cansleep(leds->desc[i], i < (unsigned int)level);
    mutex_unlock(&led_lock);
    return 0;
}
```

```
fan_led {
    compatible = "kcci,fan-led";
    led-gpios = <&gpio ((18 * 8) + 0) 0>,
                <&gpio ((18 * 8) + 4) 0>,
                <&gpio ((18 * 8) + 3) 0>,
                <&gpio ((18 * 8) + 1) 0>;
    pinctrl-names = "default";
    pinctrl-0 = <&fan_led_pins>;
};
```

devm_gpiod_get_array로 led-gpios 획득

0–4 범위 검사 후 descriptor 배열 갱신

mutex + 단일 open으로 제어 상태 보호

driver/led/fan_led_main.c · dts/fan-overlay.dts

BMP180: 메모리 맵

BMP180을 작동시키기 위한 외부 접근 가능 레지스터

Register Name	Register Adress	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset state
out_xlsb	F8h	adc_out_xlsb<7:3>					0	0	0	00h
out_lsb	F7h	adc_out_lsb<7:0>								00h
out_msb	F6h	adc_out_msb<7:0>								80h
ctrl_meas	F4h	oss<1:0>		sco	measurement control					00h
soft reset	E0h	reset								00h
id	D0h	id<7:0>								55h
calib21 downto calib0	BFh *down* to AAh	calib21<7:0> down to calib0<7:0>								n/a

Registers:	Control registers	Calibration registers	Data registers	Fixed
Type:	read / write	read only	read only	read only

Measurement control (register F4h <4:0>): Controls measurements. Refer to Figure 6 for usage details.

Sco (register F4h <5>): Start of conversion. The value of this bit stays "1" during conversion and is reset to "0" after conversion is complete (data registers are filled).

Oss (register F4h <7:6>): controls the oversampling ratio of the pressure measurement (00b: single, 01b: 2 times, 10b: 4 times, 11b: 8 times).

Chip-id (register D0h): This value is fixed to 0x55 and can be used to check whether communication is functioning.

ctrl_meas register

- oss: 압력 측정의 오버샘플링 비율 설정 (본 프로젝트 00b: single)
- sco: 변환 중 1, 완료 0

id register

- 0x55 값 고정 → 통신 확인 용도로 사용 가능

driver/bmp180/fan_bmp180_sensor.c:10–20

BMP180: 초기화

드라이버 초기화 코드

Chip-id (register D0h): This value is fixed to 0x55 and can be used to check whether communication is functioning.

```
chip_id = i2c_smbus_read_byte_data(client, BMP180_CHIP_ID_REG);

if (chip_id != BMP180_CHIP_ID) // 0x55
    return -ENODEV;
```

Chip ID 0x55 대조 검증
(장치 및 통신 무결성 확인)

Read calibration data from the E²PROM of the BMP180	
read out E²PROM registers, 16 bit, MSB first	
AC1 (0xAA, 0xAB)	(16 bit)
AC2 (0xAC, 0xAD)	(16 bit)
AC3 (0xAE, 0xAF)	(16 bit)
AC4 (0xB0, 0xB1)	(16 bit)
AC5 (0xB2, 0xB3)	(16 bit)
AC6 (0xB4, 0xB5)	(16 bit)
B1 (0xB6, 0xB7)	(16 bit)
B2 (0xB8, 0xB9)	(16 bit)
MB (0xBa, 0xBB)	(16 bit)
MC (0xBC, 0xBD)	(16 bit)
MD (0xBE, 0xBF)	(16 bit)

EEPROM 보정 계수 11개

```
struct fan_bmp180_calibration {
    s16 ac1;
    s16 ac2;
    s16 ac3;
    u16 ac4;
    u16 ac5;
    u16 ac6;
    s16 b1;
    s16 b2;
    s16 mb;
    s16 mc;
    s16 md;
};
```

구조체 선언

```
    c->ac1 = (s16)words[0];
    c->ac2 = (s16)words[1];
    c->ac3 = (s16)words[2];
    c->ac4 = words[3];
    c->ac5 = words[4];
    c->ac6 = words[5];
    c->b1  = (s16)words[6];
    c->b2  = (s16)words[7];
    c->mb  = (s16)words[8];
    c->mc  = (s16)words[9];
    c->md  = (s16)words[10];
```

워드 (22바이트) 초기화

driver/bmp180/fan_bmp180_sensor.c:44–82, 355–365

BMP180: 변환 완료와 오류 처리

명령 전송 → SCO 확인 → 원시값 읽기

Sco (register F4h <5>): Start of conversion. The value of this bit stays "1" during conversion and is reset to "0" after conversion is complete (data registers are filled).

```
ret = read_poll_timeout(i2c_smbus_read_byte_data, status,
                        status < 0 || !(status & BMP180_SCO),
                        BMP180_POLL_US,
                        conversion_us + BMP180_TIMEOUT_MARGIN_US,
                        false, data->client, BMP180_CTRL_MEAS_REG);
if (status < 0)
    return status;

return ret;
```

SCO 비트(bit 5)를 sleep 폴링해 변환 완료 감지

Table 8: Control registers values for different internal oversampling_setting (oss)

Measurement	Control register value (register address 0xF4)	Max. conversion time [ms]
Temperature	0x2E	4.5
Pressure (oss = 0)	0x34	4.5
Pressure (oss = 1)	0x74	7.5
Pressure (oss = 2)	0xB4	13.5
Pressure (oss = 3)	0xF4	25.5

온도 4.5ms / 기압 4.5ms 변환 시간 준수

driver/bmp180/fan_bmp180_sensor.c:32–40

BMP180: 정수 보정과 단위

원시 ADC 값 → 보정 온도 → 앱의 0.1°C 값

```c
x1 = (((s64)ut - c->ac6) * c->ac5) >> 15;

    denominator = x1 + c->md;
    if (!denominator)
        return -ERANGE;

    x2 = div64_s64((s64)c->mc * 2048, denominator);
    b5 = x1 + x2;

    /* 0.1°C 단위의 결과를 m°C로 변환 */
    temperature = ((b5 + 8) >> 4) * 100;
```

calculate true temperature
$X1 = (UT - AC6) * AC5 / 2^{15}$
$X2 = MC * 2^{11} / (X1 + MD)$
$B5 = X1 + X2$
$T = (B5 + 8) / 2^4$

계수의 signed/unsigned와
64비트 중간값 0으로 나누기·결과 범위 오류 검사

ex. 내부 24,600 m°C → 장치 문자열 246 → 24.6°C

driver/bmp180/fan_bmp180_sensor.c:167–177

BMP180: I²C 드라이버 통신

온도 측정 명령 → 변환 대기 → 결과 2바이트 읽기

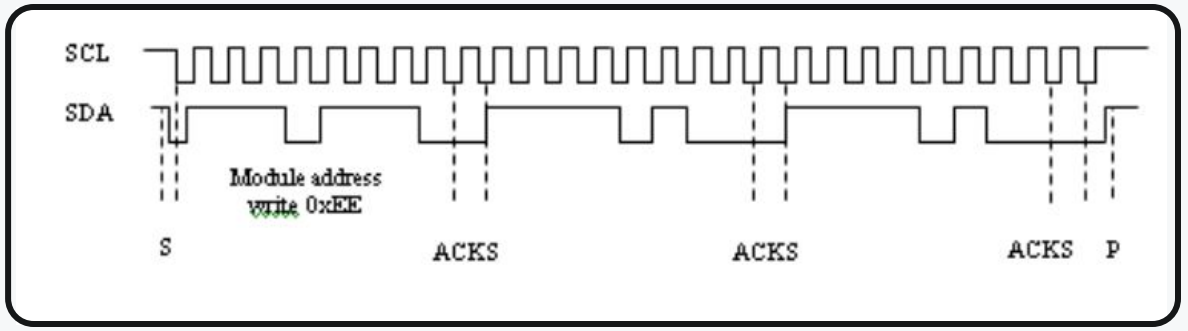
```
    ret = i2c_smbus_write_byte_data(data->client,
                                    BMP180_CTRL_MEAS_REG,
                                    BMP180_TEMP_CMD);

    ret = fan_bmp180_wait_conversion(data, 4500);

    for (i = 0; i < ARRAY_SIZE(buf); i++) {
        ret = i2c_smbus_read_byte_data(data->client,
        BMP180_OUT_MSB_REG + i);
        buf[i] = (u8)ret;
    }

    *ut = ((u16)buf[0] << 8) | buf[1];
```

```
fragment@1 {
    target-path = "/bus@0/i2c@c250000";
    __overlay__ {
        #address-cells = <1>;
        #size-cells = <0>;
        bmp180@77 { compatible = "kcci,bmp180"; reg = <0x77>; };
    };
};
```



I²C 마스터가 슬레이브 장치의 제어 레지스터(0xF4)로 온도 측정 시작 커맨드 1바이트를 전송

SCO 비트가 0으로 내려갈 때까지 최대 4.5ms 동안 대기

I²C 버스를 통해 센서 내부의 결과 레지스터에서 연속된 2바이트를 순서대로 읽어옴

i = 0: BMP180_OUT_MSB_REG (0xF6, MSB 8비트)
i = 1: BMP180_OUT_MSB_REG + 1 (0xF7, LSB 8비트)

driver/bmp180/fan_bmp180_sensor.c:95–114

센서 파일 읽기의 일관성

한 번의 측정 결과를 파일별 버퍼에 보관

- 요청한 버퍼가 작거나 커널 구현에 따라 파일 전체 내용 "246" 중 일부 바이트만 먼저 읽히는 경우
- 첫 번째 read에서 "24"를 읽고
- 두 번째 read에서 센서를 다시 측정해 값이 "251"로 바뀌면 나머지 "1"이 읽혀
- 두 측정이 섞인 "241"이 되는 데이터 불일치 문제가 발생

최초 read에서 측정하고 문자열 캐시 생성

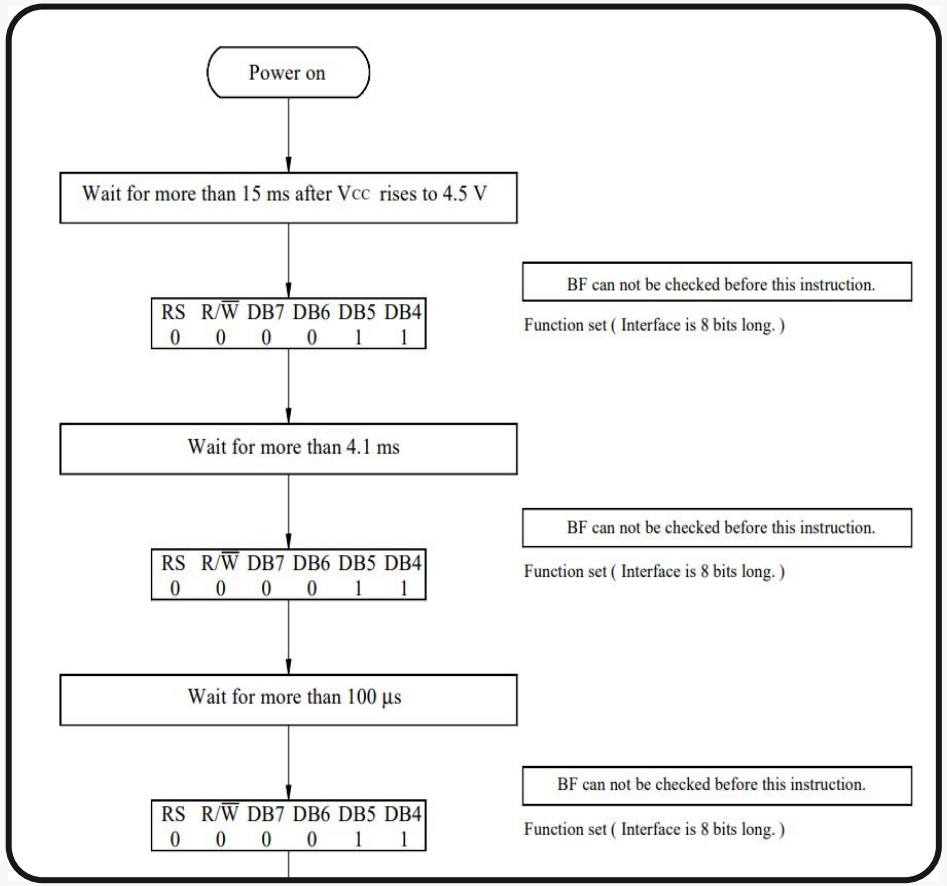
부분 read는 같은 샘플을 이어서 반환

lseek(fd, 0, SEEK_SET)으로 다음 측정 시작

driver/bmp180/fan_bmp180_device.c · app/src/bmp180.c

LCD1602: 4비트 모드 초기화

하드웨어 초기화




```c
ret = fan_lcd_output(data, data->backlight ? LCD_BACKLIGHT_MASK : 0);
    if (ret)
        return ret;
    msleep(50);

    ret = fan_lcd_write_nibble(data, 0x30, false);
    if (ret)
        return ret;
    msleep(5);

    ret = fan_lcd_write_nibble(data, 0x30, false);
    if (ret)
        return ret;
    usleep_range(200, 300);

    ret = fan_lcd_write_nibble(data, 0x30, false);
    if (ret)
        return ret;

    ret = fan_lcd_write_nibble(data, 0x20, false);
    if (ret)
        return ret;

    for (i = 0; i < ARRAY_SIZE(commands); i++) {
        ret = fan_lcd_command(data, commands[i]);
        if (ret)
            return ret;
    }
```

driver/lcd1602/fan_lcd_hw.c:105–132

LCD1602: I²C에서 4비트 버스로

PCF8574 핀 조합으로 RS·E·DB4–DB7 제어

```c
static int fan_lcd_write_byte(struct fan_lcd_data *data, u8 value, bool rs)
{
    int ret;

    ret = fan_lcd_write_nibble(data, value, rs);
    if (ret)
        return ret;

    return fan_lcd_write_nibble(data, value << 4, rs);
}
```

RS: 명령 / 데이터
E: 데이터 전달 펄스

상위 nibble → 하위 nibble 순서로 전송
초기화: 0x30 반복 → 0x20 → 4비트 모드

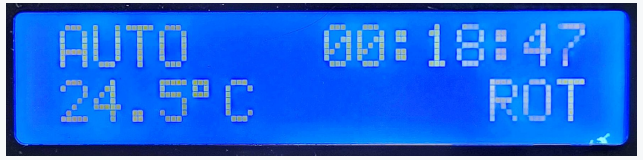
driver/lcd1602/fan_lcd_hw.c:47–56

LCD 주소와 파일 위치

32칸 논리 공간을 두 줄의 DDRAM 주소로 매핑

```c
int fan_lcd_set_cursor(struct fan_lcd_data *data, unsigned int offset)
{
    if (offset ≥ FAN_LCD_SIZE)
        return -EINVAL;
    return fan_lcd_command(data, (offset < LCD_COLUMNS ? LCD_ROW1_ADDR :
                           LCD_ROW2_ADDR) + offset % LCD_COLUMNS);
}
```





offset 0–15 → 명령 0x80–0x8F

offset 16–31 → 명령 0xC0–0xCF

pwrite로 모드·온도·타이머·회전 영역 지정

모드	타이머
온도	회전

driver/lcd1602/fan_lcd_hw.c:138–143

앱 동시성과 하드웨어 반영

공유 상태를 복사한 뒤 mutex 밖에서 장치 I/O

```
copy_run_value = g_ctx;

pthread_mutex_unlock(&g_state_lock);

/* 하드웨어 반영 */
if(hw_flag) apply_to_hardware(&copy_run_value);

/* 100ms 대기 (I/O 폭주 및 CPU 100% 점유 방지) */
usleep(100000);
```

Input 80 · Timer 50 · Sensor 20: SCHED_FIFO 요청

입력·센서: 공유 상태 수정

타이머: 상태 snapshot → 출력 적용 → 100 ms 대기

app/src/thread.c:106–114

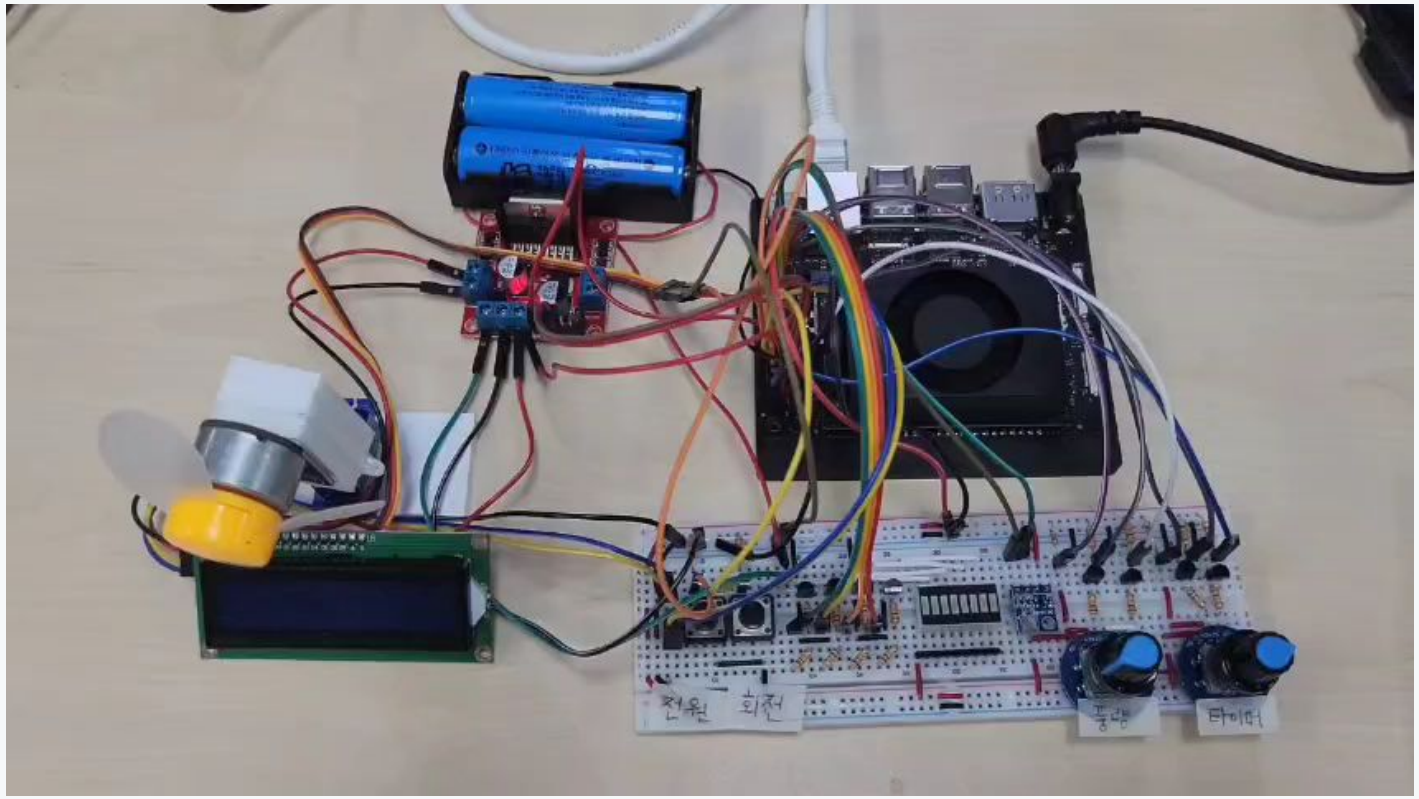
통합 검증과 시연 시나리오

구현 설명에 실제 로그·파형·동작 영상을 연결

검증 대상	관찰할 근거	판정 포인트
입력	이벤트 로그·회전 횟수	detent당 ±1 · overflow 확인
모터 / 서보	PWM 파형·구동 영상	20 kHz · pulse width · 기동/정지
센서 / LCD	온도 로그·표시 영상	단위 · 새 샘플 · 줄 경계
통합	전원·회전·타이머 시연	명령 상태와 실제 출력의 일치
오류·종료	오류 로그·출력 상태	버스 오류·닫기·제거 정책

app/tests/bmp180_test.c · app/tests/lcd_device_test.c

동작 동영상



- 전원 ON/OFF
- 회전 버튼으로 서보 스윙
- 풍량 1~4단계 조절
- 타이머 설정(10분 단위), 만료 시 전원 OFF
- AUTO 모드에서 온도에 따른 풍량
- LCD, LED 표시 변화

형상 관리

GitHub


Merge pull request #10 from Electric-fan-driver/feat/input-buttons shinyuji
origin/main
AUTO 정책이 온도를 0.1도 단위로 비교하도록 수정 (소수점 버림 문제) yuji
AUTO 모드 0단계 추가, 타이머를 돌린 순간 바로 표시하고 깜빡임 주기 단축 yuji
전원 OFF일 때 LCD 백라이트를 끄도록 수정 yuji
load_all.sh가 입력 모듈 옵션을 넘기도록 수정 yuji
이벤트 큐를 kfifo_reset 대신 INIT_KFIFO로 초기화 yuji
button_input.c에 빠진 linux/of.h 포함 추가 yuji
입력 모듈에 전원,회전 버튼 입력 추가 yuji
Incoming Changes origin/main
Merge branch 'main' of https://github.com/Electric-fan-driver/Fan_Linux_Driver rlagudwls5518
main
Merge pull request #9 from Electric-fan-driver/fix/build Do Jae Woo
Merge branch 'main' into fix/build Do Jae Woo
fix: 드라이버·앱 빌드 및 모듈 적재 흐름 정리 Kevin-jwd
fix: 파일 삭제 rlagudwls5518
build: 빌드파일 수정 rlagudwls5518
build: makefile 수정중 rlagudwls5518
Merge pull request #8 from Electric-fan-driver/fix/driver-of-header shinyuji
linux/of.h 추가 yuji
docs: README를 현재 프로젝트 구현에 맞게 수정 Do Jae Woo
fix: 빈파일 삭제 rlagudwls5518
chore: 타이머 파일 폴더변경 rlagudwls5518
Merge pull request #2 from Electric-fan-driver/feat/app 김형진
Merge branch 'feat/app' of https://github.com/Electric-fan-driver/Fan_Linux_Driver into feat/app rlagudwls5518
Merge branch 'main' into feat/app 김형진
build: 드라이버 모듈 파일 수정 rlagudwls5518
Merge remote-tracking branch 'origin/main' into feat/app rlagudwls5518
Merge pull request #5 from Electric-fan-driver/feat/motor-servo-driver shinyuji
refactor: 모터,서보 드라이버를 main 규칙에 맞춰 장치별 폴더로 이동 yuji
feat: 모터·서보 드라이버와 motor.c 구현 yuji
Merge pull request #6 from Electric-fan-driver/feat/overlay-motor-servo shinyuji
feat: 오버레이에 서보 주기, 모터,LED,버튼 핀 설정 추가 yuji
Merge pull request #7 from Electric-fan-driver/feat/led-driver shinyuji
feat: LED 드라이버와 led.c 구현 yuji
Merge pull request #4 from Electric-fan-driver/feat/encoder-timer 김형진
Merge remote-tracking branch 'origin/main' into feat/app rlagudwls5518
Merge pull request #3 from Electric-fan-driver/refactor/driver-layout Do Jae Woo
docs: bmp180, lcd1602 하드웨어 핀 맵 수정 Kevin-jwd
docs: IIO 인터페이스 내용 제거 Kevin-jwd
docs: README 프로젝트 구조 최신화 Kevin-jwd
refactor: BMP180과 LCD 드라이버를 장치별 폴더로 정리 Kevin-jwd
feat: BMP180 실제 온도를 LCD에 표시하는 데모 추가 Kevin-jwd

트러블 슈팅

모터 미동작

원인 젯슨 5v 출력 고장

해결 외부 배터리 연결

신호 약함

원인 LED 흐림, 엔코더 인식 불가

해결 NPN 트랜지스터, 외부 풀업 추가

LCD 무응답

원인 5V 모듈과 3.3V 신호 레벨 불일치

해결 회로 결선 변경

통합 단계 충돌

원인 큐 초기화 누락, 빌드 파일 충돌

해결 젯슨에서 직접 빌드, 로그로 원인 추적

프로젝트 수행 후기

코드보다 하드웨어 설정에서 더 많이 막힘

- 핀 tristate, 신호 레벨, 전원 문제
- 코드가 맞아도 동작 안 함
- 멀티미터로 하나씩 확인해서 문제 해결

공용 헤더를 먼저 정한 것이 통합에 도움

- fan_ioctl.h, fan_api.h로 통일
- 4명이 나눠 개발해도 연결이 맞음
- PR로 변경 내용을 서로 확인

감사합니다

